

# Tool use

## Learning calls and executing actions

CMSC 848N Fall 2026  
Week 5, Lecture 2 October 1  
Furong Huang

# A task with a checkable answer

Find the run with the highest validation accuracy among runs that use at most 8 GB. Report its ID and accuracy.

| Run | Validation accuracy | Memory (GB) |
|-----|---------------------|-------------|
| A   | 0.81                | 6           |
| B   | 0.84                | 10          |
| C   | 0.83                | 8           |

Synthetic records for a classroom example

# The model reaches the records through tools

```
list_runs()  
    returns ["A", "B", "C"]
```

```
get_metrics(run_id: str)  
    returns {"accuracy": float, "memory_gb": float}
```

The prompt contains the task and tool descriptions.  
The records arrive only after the harness executes calls.

# One complete execution trace

| Step | Requested action | Returned observation        |
|------|------------------|-----------------------------|
| 1    | list_runs()      | [A, B, C]                   |
| 2    | get_metrics("A") | accuracy 0.81, memory 6 GB  |
| 3    | get_metrics("B") | accuracy 0.84, memory 10 GB |
| 4    | get_metrics("C") | accuracy 0.83, memory 8 GB  |

**Final answer: C, validation accuracy 0.83**

# Recovery depends on the failed operation

## Observation

```
get_metrics("C")  
TimeoutError("metrics service did not reply")
```

**A bounded retry can recover this read-only query.**

If the retry also fails, the agent cannot establish the best run from the available records.

# ReAct conditions each action on the trace

$$h_t = (q, a_1, o_1, \dots, a_{t-1}, o_{t-1}), \quad a_t \sim \pi_\theta(\cdot \mid h_t)$$

$q$  is the user request.  $a$  is an action.  $o$  is a tool result.

ReAct inserts reasoning text between actions. That text can propose a step or interpret an observation.

**A useful trace must still end in a correct, verifiable result.**

# The harness owns the execution loop

```
for step in range(call_budget):
    reply = model(history, tool_schemas)
    if reply.is_final:
        return reply.text
    call = parse_call(reply)
    tool = registry.lookup(call.name)
    args = tool.validate(call.arguments)
    policy.authorize(tool, args)
    result = tool.execute(args)
    history.append(tool_result(call.id, result))
```

**A model proposes a call. Software decides whether and how to run it.**

# Toolformer builds tool-use training data

Training data can include tool calls and their results.  
Where do those examples come from?

## Toolformer's construction

Sample candidate calls at useful positions.  
Execute each candidate.  
Keep calls whose responses improve continuation prediction.  
Fine-tune on the augmented text.

# Sample likely positions for a tool call

$$p_i = p_M(\langle \text{API} \rangle \mid P(x), x_{1:i-1})$$

$P(x)$  is a prompt that demonstrates the tool.  
 $i$  identifies a position in the original text.

Keep positions above a threshold, retain at most  $k$ , and sample candidate calls at those positions.

# A candidate supplies an extra context

## Original text

Seven items cost \$12 each. The total is 84 dollars.

## Candidate call and observed result

```
[Calculator(7 * 12) -> 84]
```

The filter asks whether this extra context helps the model predict the original continuation.

# Score the original continuation

$$L_i(z) = - \sum_{j=i}^n w_{j-i} \log p_M(x_j \mid z, x_{1:j-1})$$

| Symbol          | Meaning                                      |
|-----------------|----------------------------------------------|
| $z$             | Extra prefix: a call, a response, or nothing |
| $x_i \dots x_n$ | Original continuation tokens                 |
| $w_{j-i}$       | Nonnegative weights on nearby tokens         |

# Compare with the stronger baseline

$$L_i^+ = L_i(e(c_i, r_i))$$

Call plus its result

$$L_i^- = \min\{L_i(\varepsilon), L_i(e(c_i, \varepsilon))\}$$

Best baseline: no call, or the call without its result

# Keep calls that reduce prediction loss

$$\Delta_i = L_i^- - L_i^+, \quad \text{keep the call if } \Delta_i \geq \tau_f$$

A positive gain means the response reduces prediction loss relative to both baselines.

The threshold controls which proposed annotations enter training.

# A numerical filtering example

One target token, weight 1. These probabilities are illustrative.

| Scoring context          | P(correct token) | $-\log P$ |
|--------------------------|------------------|-----------|
| No call                  | 0.20             | 1.609     |
| Call without result      | 0.25             | 1.386     |
| Call plus correct result | 0.80             | 0.223     |

$$\Delta = \min(1.609, 1.386) - 0.223 = 1.163 \geq 0.5$$

# A weak response contributes almost nothing

Keep the same baselines, but let the response raise the target probability only to 0.26.

$$L^+ = -\log(0.26) = 1.347$$

$$\Delta = 1.386 - 1.347 = 0.039 < 0.5$$

**Reject this candidate annotation.**

# Fine-tune on the accepted calls

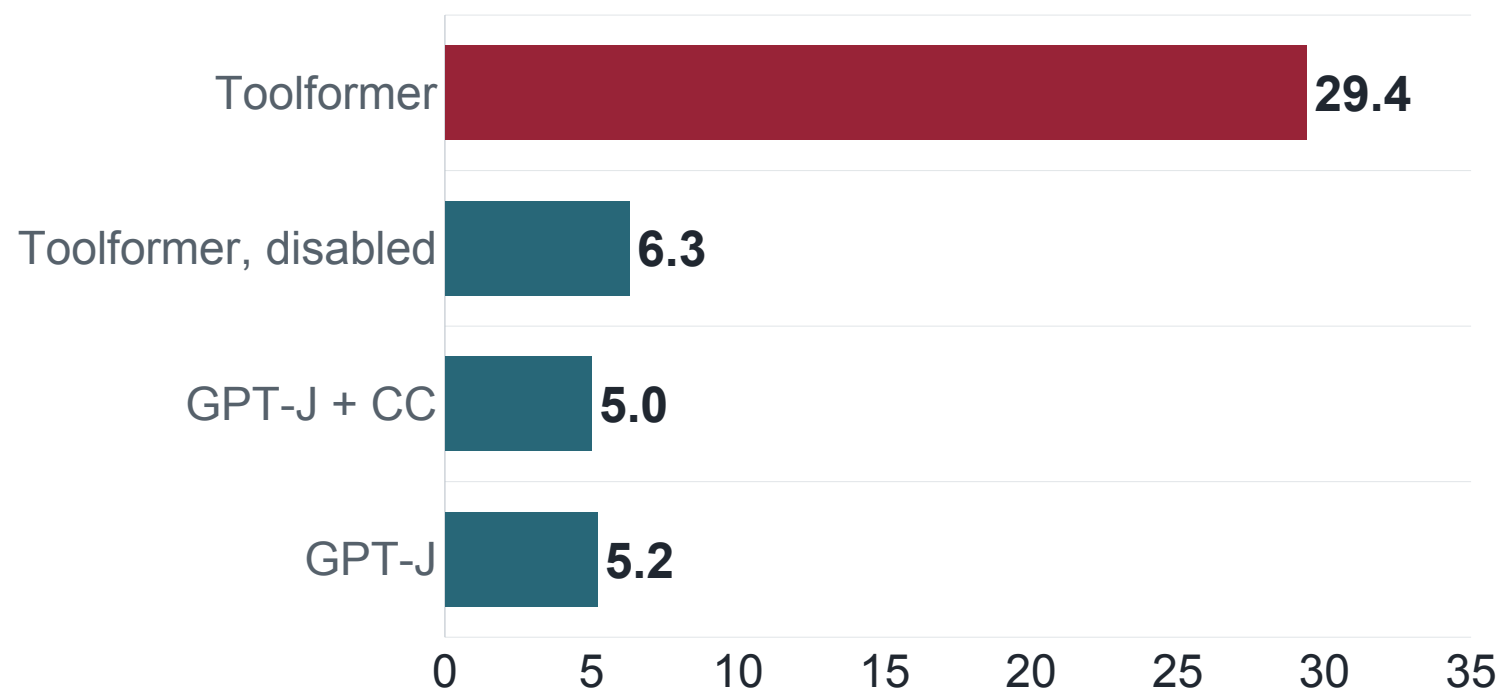
$$x^* = (x_{1:i-1}, e(c_i, r_i), x_{i:n})$$

Fine-tuning uses a language-model objective on augmented sequences.

At inference, generation pauses for the tool result and then continues with that result in context.

# Tool execution matters after fine-tuning

## SVAMP accuracy (%)



6.7B GPT-J backbone

Published 2023 setup

Calls enabled: 29.4%

Calls disabled: 6.3%

**The enabled/disabled comparison holds the fine-tuned model fixed.**

Toolformer, Table 4, SVAMP column. Historical result.

# Prediction gain is an incomplete tool objective

## Useful signal

The returned information makes nearby text easier to predict.

## Unanswered questions

Does the call help finish a multi-step task?

Is its latency or monetary cost justified?

Could it change external state without authorization?

# Code expresses the comparison directly

```
rows = []
for run_id in list_runs():
    m = get_metrics(run_id)
    if m["memory_gb"] <= 8:
        rows.append((run_id, m["accuracy"]))

if not rows:
    raise ValueError("no eligible run")
best = max(rows, key=lambda row: row[1])
print(best)
```

**Output for the frozen fixture: ("C", 0.83)**

# Where does the control flow live?

| Decision             | Individual tool calls              | Executable program                 |
|----------------------|------------------------------------|------------------------------------|
| Read each run        | Model or harness schedules calls   | A loop schedules calls             |
| Keep eligible runs   | Model or a dedicated tool filters  | A condition filters                |
| Compare accuracy     | Model or a dedicated tool compares | max(...) compares                  |
| Recover from failure | Another model turn or harness rule | Exception handling or another turn |

**A JSON interface can also support parallel calls or expose a combined operation.**

# Execution errors can guide a later revision

## First program

```
score = metrics["validation_accuracy"]
```

## Observation

```
KeyError: "validation_accuracy"  
Available fields: accuracy, memory_gb
```

## Revised program

```
score = metrics["accuracy"]
```

# CodeAct's advantage depends on the setting

| Model and benchmark                         | CodeAct | JSON |
|---------------------------------------------|---------|------|
| GPT-4-1106-preview, API-Bank                | 76.7    | 82.7 |
| GPT-4-1106-preview, M <sup>3</sup> ToolEval | 74.4    | 52.4 |
| Claude-instant-1, M <sup>3</sup> ToolEval   | 20.7    | 31.7 |

Reported correctness or success rate (%).

M<sup>3</sup>ToolEval contains 82 human-curated multi-tool tasks.

**These selected rows show why one format cannot claim a universal win.**

# Compare both interfaces on the same tasks

Compare a JSON-calling agent with a code-action agent on the same frozen tasks.

| Hold fixed                            | Measure                                       |
|---------------------------------------|-----------------------------------------------|
| Model version and available functions | Task success under the stated constraint      |
| Data snapshot and task instructions   | Underlying calls, including calls inside code |
| Token, call and time budgets          | Latency, token use and unauthorized writes    |

# Identify the tool, arguments and request

```
{  
  "id": "call_7",  
  "name": "get_metrics",  
  "arguments": {"run_id": "C"}  
}
```

The registry resolves the name.

The schema checks the arguments.

The call ID connects the result to the request.

# A schema makes the argument contract explicit

```
{  
  "type": "object",  
  "properties": {  
    "run_id": {"type": "string", "enum": ["A", "B", "C"]}  
  },  
  "required": ["run_id"],  
  "additionalProperties": false  
}
```

For this frozen fixture, the allowed IDs are known.  
A live service needs a way to update or validate those IDs.

# Mask tokens that violate the constraint

$$p_C(v \mid y_{<t}) = \frac{p_\theta(v \mid y_{<t}) \mathbf{1}[v \in V_C(y_{<t})]}{\sum_{u \in V_C(y_{<t})} p_\theta(u \mid y_{<t})}$$

$V_C$  contains tokens that preserve a valid continuation under the constraint.

The implementation masks invalid logits before sampling.  
It must handle an empty valid set as an error.

# One token can cross several grammar elements

Current prefix

```
{"run_id":
```

Illustrative next token

```
C"}
```

The checker consumes C, the closing quote and the closing brace.  
Token boundaries can cross a value and its enclosing object.

# Check the final answer against the task

```
{"run_id": "B"}
```

Valid name and argument. Existing record.  
Memory use: 10 GB.

**The final recommendation must still satisfy the user's 8 GB limit.**

# A timeout can leave a write's outcome unknown

## Read-only request

```
get_metrics("C")
```

A bounded retry reads the frozen record again.

## Request that starts a job

```
start_job("C")
```

The service may have started the job before the reply was lost.

**A retry needs an operation ID or a service-supported idempotency key.**

# MCP standardizes discovery and invocation

## Discover tools

```
{"jsonrpc":"2.0", "id":1, "method":"tools/list"}
```

## Invoke a discovered tool

```
{"jsonrpc":"2.0", "id":2, "method":"tools/call",  
  "params":{"name":"get_metrics",  
            "arguments":{"run_id":"C"}}}
```

**The server still needs access checks and correct tool implementations.**

# Turn the task requirements into tests

| Fixture                                   | Expected behavior                |
|-------------------------------------------|----------------------------------|
| A: 0.81/6 GB; B: 0.84/10 GB; C: 0.83/8 GB | Return C with accuracy 0.83      |
| Every run uses more than 8 GB             | Report no eligible run           |
| C times out once, then returns normally   | Retry within budget and return C |
| C never returns its metrics               | Report incomplete evidence       |
| Tool name or argument is invalid          | Reject before execution          |

# Starting a job needs permission

“Now start a new training job for the selected run.”

Who may start it?

Which account pays for the compute?

What budget should the system enforce?

Later topic: authorization, untrusted tool output and information flow.

# Readings and questions

**Toolformer, Schick et al. (2023)**

Section 2: Which calls survive filtering, and why?

**CodeAct, Wang et al. (2024)**

Section 2: What changes when actions are programs?

**Further reading: CaMeL, Debenedetti et al. (2025)**

Follow the document-sharing example and its security policy.

# Appendix: Toolformer's nearby-token weights

$$\tilde{w}_t = \max(0, 1 - 0.2t), \quad w_t = \frac{\tilde{w}_t}{\sum_{s \geq 0} \tilde{w}_s}$$

| Offset <b>t</b>     | <b>0</b> | <b>1</b> | <b>2</b> | <b>3</b> | <b>4</b> |
|---------------------|----------|----------|----------|----------|----------|
| Unnormalized weight | 1.0      | 0.8      | 0.6      | 0.4      | 0.2      |
| Normalized weight   | 1/3      | 4/15     | 1/5      | 2/15     | 1/15     |

Later offsets receive zero weight under this schedule.

# Appendix: a masking calculation

Illustrative vocabulary fragment. Valid tokens are A and C.

| Token | Original probability | Allowed? | After masking       |
|-------|----------------------|----------|---------------------|
| A     | 0.20                 | Yes      | $0.20 / 0.30 = 2/3$ |
| B     | 0.70                 | No       | 0                   |
| C     | 0.10                 | Yes      | $0.10 / 0.30 = 1/3$ |

**Masking removes invalid mass and renormalizes what remains.**

# Appendix: when the call already gives the answer

Illustrative target probability without the result: 0.90.

With the result: 0.91. No-call probability: 0.20.

$$L^- = \min(-\log 0.20, -\log 0.90) = -\log 0.90$$

$$\Delta = \log(0.91/0.90) \approx 0.011 < 0.5$$

# Appendix: report the work done and failures

For each task, retain the exact request, tool schema, data snapshot and execution trace.

| Outcome                      | Reason to record it                         |
|------------------------------|---------------------------------------------|
| Correctness                  | The answer can name the wrong run           |
| Calls and time               | One program can perform many operations     |
| Errors and policy violations | A correct answer can hide unsafe execution  |
| Variation across trials      | One trace gives a weak reliability estimate |